

Activer Strava.

Le code est écrit et testé. Il reste quatre étapes, toutes en dehors du dépôt. Compter vingt minutes.

app.colosmartraining.fr · Supabase `lupqpeqvtxnohjpolhna`

Où on en est. Toute la mécanique est déjà codée : connexion du membre à son compte Strava, rafraîchissement automatique des jetons, import des sorties, webhook temps réel, rattachement à la séance de course du jour, écrans côté coach et côté membre.

Rien n'a jamais tourné pour une seule raison : **les deux tables n'ont jamais été créées en production**. La migration existe dans le dépôt depuis juillet, elle n'a pas été appliquée.

01 Créer les deux tables

La production est hors MCP : le SQL se colle à la main.

- 1 Ouvrir le **dashboard Supabase** du projet `lupqpeqvtxnohjpolhna` → **SQL Editor**.
- 2 Coller tout le contenu de `supabase/migrations/20260727170000_add_member_strava_sync.sql` (il est aussi reproduit en annexe de ce document).
- 3 Exécuter.

SANS RISQUE

Le script est **idempotent** : `create table if not exists` et politiques conditionnelles. Le relancer deux fois ne casse rien et ne duplique rien.

TABLE	RÔLE
member_strava_connections	Un membre ↔ son compte Strava : jetons, expiration, autorisations, dates de synchro.
member_strava_activities	Les sorties importées, et la séance à laquelle chacune est rattachée.

Avec les index et les politiques de sécurité (le membre lit les siennes, le coach lit celles de ses athlètes).

Vérifier

```
select count(*) from public.member_strava_connections;
select count(*) from public.member_strava_activities;
```

Deux 0 : c'est bon.

02 Créer l'application Strava

Sur **strava.com/settings/api**, avec le compte Strava qui portera l'application :

CHAMP	VALEUR
Application Name	ColoSmart Training
Category	Training
Website	<code>https://app.colosmartraining.fr</code>
Authorization Callback Domain	<code>app.colosmartraining.fr</code>

LE PIÈGE CLASSIQUE

Le champ **Authorization Callback Domain** attend **le domaine seul** : pas de `https://` , pas de chemin, pas de barre oblique. C'est ce qui fait échouer la connexion avec une erreur « `redirect_uri invalide` ».

Strava affiche ensuite un **Client ID** et un **Client Secret** : garde-les pour l'étape suivante.

03 Renseigner les clés sur Vercel

Projet `colo-smart-app` → **Settings** → **Environment Variables**. À créer pour **Production** (et Preview si tu veux tester sur les previews) :

VARIABLE	VALEUR
<code>STRAVA_CLIENT_ID</code>	le Client ID donné par Strava
<code>STRAVA_CLIENT_SECRET</code>	le Client Secret donné par Strava
<code>STRAVA_STATE_SECRET</code>	une phrase secrète que tu génères
<code>STRAVA_WEBHOOK_VERIFY_TOKEN</code>	une seconde phrase secrète que tu génères
<code>APP_URL</code>	<code>https://app.colosmartraining.fr</code> (si absente)

`STRAVA_STATE_SECRET` signe le jeton d'état de la connexion : il empêche qu'un compte Strava soit rattaché au mauvais membre. `STRAVA_WEBHOOK_VERIFY_TOKEN` sert uniquement à ce que Strava prouve son identité au moment de s'abonner.

```
openssl rand -hex 32
openssl rand -hex 32
```

À NE PAS OUBLIER

Redéployer après avoir ajouté les variables : elles ne sont lues qu'au démarrage de l'application.

04 Abonner le webhook

C'est ce qui fait remonter les sorties **automatiquement**, sans que le membre ouvre l'app. À lancer **une seule fois**, une fois le site redéployé :

```
curl -X POST https://www.strava.com/api/v3/push_subscriptions \
-F client_id=TON_CLIENT_ID \
-F client_secret=TON_CLIENT_SECRET \
-F callback_url=https://app.colosmartraining.fr/api/strava/webhook \
-F verify_token=TON_STRAVA_WEBHOOK_VERIFY_TOKEN
```

Strava appelle immédiatement l'URL pour la valider, puis renvoie un identifiant d'abonnement. Une erreur ici veut presque toujours dire que le site n'a pas encore été redéployé avec `STRAVA_WEBHOOK_VERIFY_TOKEN`.

Vérifier l'abonnement

```
curl -G https://www.strava.com/api/v3/push_subscriptions \
-d client_id=TON_CLIENT_ID -d client_secret=TON_CLIENT_SECRET
```

05 Tester

- 1 Se connecter en **coaché** → Réglages → Connexions → **Connecter Strava**.
- 2 Autoriser sur Strava (autorisations demandées : `read` , `activity:read_all`).
- 3 De retour dans l'app : « **Compte Strava connecté** », avec l'identifiant athlète et la date de dernière synchro.
- 4 Enregistrer une sortie sur Strava, ou en modifier une : elle doit apparaître dans **Trail & Run** en moins d'une minute, sans rien faire.

06 Ce que ça donne

- Les sorties **course à pied et trail** remontent seules : distance, temps, dénivelé, fréquence cardiaque moyenne, vitesse moyenne.
- Chaque sortie se **rattache à la séance de course du jour** quand il y en a une : le coach voit le réalisé en face du prescrit.
- Les jetons se **rafraîchissent seuls** (ils expirent toutes les six heures).
- Le membre peut **déconnecter** son compte quand il veut, au même endroit.

Limites connues

- Seuls `Run` et `TrailRun` sont importés. Vélo, natation et renfo sont ignorés volontairement.
- L'abonnement webhook est **unique par application Strava** : pour le faire pointer ailleurs, supprimer l'abonnement existant (`DELETE /push_subscriptions/{id}`) avant d'en créer un autre.

A Annexe — le SQL à coller

```
-- Connexion Strava individuelle par membre + journal d'import des activités.
```

```
create table if not exists public.member_strava_connections (  
  member_id uuid primary key references public.profiles(id) on delete cascade,  
  strava_athlete_id bigint not null unique,  
  access_token text not null,  
  refresh_token text not null,  
  expires_at timestamptz not null,  
  scope text[] not null default '{}',  
  last_sync_at timestamptz,  
  last_webhook_at timestamptz,  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now()  
);
```

```
create table if not exists public.member_strava_activities (  
  id uuid primary key default gen_random_uuid(),  
  member_id uuid not null references public.profiles(id) on delete cascade,  
  strava_activity_id bigint not null unique,  
  session_id uuid references public.sessions(id) on delete set null,  
  activity_type text not null,  
  name text,  
  started_at timestamptz not null,  
  distance_m numeric,  
  moving_time_s integer,  
  elapsed_time_s integer,  
  elevation_gain_m integer,  
  average_hearttrate numeric,  
  average_speed_mps numeric,  
  raw_payload jsonb,  
  sync_status text not null default 'imported',  
  sync_error text,  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now()  
);
```

```
create index if not exists member_strava_activities_member_idx  
  on public.member_strava_activities(member_id, started_at desc);
```

```
create index if not exists member_strava_activities_session_idx  
  on public.member_strava_activities(session_id);
```

```
create index if not exists member_strava_connections_athlete_idx
```

```

on public.member_strava_connections(strava_athlete_id);

alter table public.member_strava_connections enable row level security;
alter table public.member_strava_activities enable row level security;

do $$
begin
    if not exists (
        select 1 from pg_policies
        where schemaname = 'public'
            and tablename = 'member_strava_connections'
            and policyname = 'Member reads own Strava connection'
    ) then
        create policy "Member reads own Strava connection"
        on public.member_strava_connections
        for select
        to authenticated
        using ((select auth.uid()) = member_id);
    end if;

    if not exists (
        select 1 from pg_policies
        where schemaname = 'public'
            and tablename = 'member_strava_connections'
            and policyname = 'Coach reads member Strava connection'
    ) then
        create policy "Coach reads member Strava connection"
        on public.member_strava_connections
        for select
        to authenticated
        using (public.has_role((select auth.uid()), 'coach'));
    end if;

    if not exists (
        select 1 from pg_policies
        where schemaname = 'public'
            and tablename = 'member_strava_activities'
            and policyname = 'Member reads own Strava activities'
    ) then
        create policy "Member reads own Strava activities"
        on public.member_strava_activities
        for select
        to authenticated
        using ((select auth.uid()) = member_id);
    end if;

    if not exists (

```

```
select 1 from pg_policies
where schemaname = 'public'
    and tablename = 'member_strava_activities'
    and policyname = 'Coach reads member Strava activities'
) then
create policy "Coach reads member Strava activities"
on public.member_strava_activities
for select
to authenticated
using (public.has_role((select auth.uid()), 'coach'));
end if;
end $$;
```